SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 3 – ARCHITECTURE / MODELING (DEVSECOPS METHODOLOGY)

Course Code:	CSA5803	**Course Title:**	DEVSECOPS ENGINEERING AND APPLICATION SECURITY
CO Assessed:	CO4 – Precision (BL1, BL2, BL3)	**Assessment:**	Assessment Tool 3
Weightage:	30% of CIA	**Total Marks:**	100
CO4 Statement:	Develop Complete DevSecOps Architecture, Methodology Documentation and GitHub Submission.		
Student Name:	Adhithyan S	**Reg. No.:**	192421102
Section / Batch:		**Date:**	

Instructions to Students

- Develop a complete and original DevSecOps architecture showing the end-to-end secure software delivery workflow.
- Include development, source control, CI/CD, automated security testing, artifact management, deployment, monitoring and feedback stages.
- Document the methodology clearly, including security controls, tools, workflow relationships and key design decisions.
- Use clear labels, consistent symbols, proper alignment and professionally formatted architecture diagrams.
- Submit the editable architecture source files, methodology documentation and an accessible GitHub repository containing all required artifacts.

Question Type	Focus Area	Questions	Marks
ARCHITECTURE / MODELING	DevSecOps Methodology	Q1	Q1 – 100
GRAND TOTAL:			**100 Marks**

DEVSECOPS

Architecture & Methodology

A Complete Reference Architecture, Delivery Methodology, and GitHub Submission Framework

Prepared for: Engineering, Security & Platform Organizations

Classification: Internal Use

Table of Contents

1. Executive Summary

This document defines a complete DevSecOps architecture and methodology for integrating security as a continuous, automated discipline across the software delivery lifecycle. It is written to serve three audiences at once: engineering leaders who need a reference architecture to plan against, delivery teams who need a concrely actionable methodology and pipeline design, and platform/security teams who need governance guardrails and a submission process for changes flowing through GitHub.

The architecture is built on the principle that security controls should be embedded as code, executed automatically inside CI/CD, and surfaced as fast, actionable feedback to developers rather than enforced as a late-stage, manual gate. Every control described here — static analysis, software composition analysis, secrets scanning, infrastructure-as-code scanning, dynamic testing, container hardening, and runtime monitoring — is mapped to a specific pipeline stage, a specific tool category, and a specific owner.

The document is organized into four parts:

- Architecture — the reference architecture, component layers, and toolchain mapped to each pipeline stage.
- Methodology — the operating model: shift-left practices, threat modeling, secure SDLC phases, compliance-as-code, and continuous feedback loops.
- Pipeline & GitHub Implementation — a concrete GitHub Actions pipeline design, branching strategy, and repository conventions.
- Governance & Submission — RACI, metrics, maturity model, and the exact GitHub submission process (branch protections, CODEOWNERS, PR templates, required checks) that operationalizes the architecture.

Adopting this framework is intended to reduce mean-time-to-remediate for vulnerabilities, eliminate manual security gate delays, and produce an auditable, versioned record of every security decision alongside the code it protects.

2. DevSecOps Reference Architecture

2.1 Guiding Principles

- Shift Left, Not Left Only — security starts at design time but continues through runtime; "shift left" is a starting point, not the whole strategy.
- Security as Code — every policy, control, and exception is expressed in a versioned, reviewable artifact (YAML/Rego/JSON), never a wiki page.
- Fail Fast, Fix Cheap — defects caught in the IDE or PR cost orders of magnitude less to fix than defects caught in production.
- Continuous Verification — a control is only real if it is automated, runs on every change, and blocks or flags reliably; manual point-in-time audits are treated as a fallback, not a primary control.
- Immutable, Auditable Pipelines — build artifacts are signed and traceable back to a specific commit, pipeline run, and set of scan results (SBOM + provenance).
- Least Privilege Everywhere — identities (human and machine), pipeline runners, and workloads are scoped to the minimum access required.
- Developer Experience First — controls that developers route around are worse than no controls; tooling must give actionable, low-noise feedback inside the workflow developers already use.

2.2 Reference Architecture — Layered View

The architecture is organized as seven layers overlaid on the standard software delivery lifecycle. Layers 1–6 map directly to lifecycle phases; Layer 7 (Governance) is cross-cutting and applies continuously.

Layer 1 — Plan & Design		
Threat Modeling (STRIDE)	**Security Requirements**	**Risk-based Story Grooming**
Layer 2 — Source & Code		
IDE Security Plugins	**Pre-commit Hooks (secrets, lint)**	**SAST on PR (CodeQL / Semgrep)**
Layer 3 — Build & Package		
SCA / SBOM (Dependency scan)	**Secrets Scanning (Gitleaks/TruffleHog)**	**Container Image Scanning (Trivy/Grype)**
Layer 4 — Test & Verify		
DAST (OWASP ZAP)	**IaC Scanning (Checkov/tfsec)**	**Policy-as-Code Gate (OPA / Conftest)**
Layer 5 — Release & Deploy		
Artifact Signing (Sigstore/Cosign)	**Admission Control (Kyverno/Gatekeeper)**	**Progressive Delivery (Canary/Blue-Green)**
Layer 6 — Operate & Monitor		
Runtime Protection (Falco/eBPF)	**SIEM / Log Correlation**	**Vulnerability & Posture Management**
Layer 7 — Governance (Cross-cutting)		

Compliance as Code (NIST/CIS/SOC2)	Audit Logging & Evidence Capture	Metrics, KPIs & Risk Dashboards

Figure 1. DevSecOps layered reference architecture, mapping controls to lifecycle phases.

2.3 Toolchain Mapping by Pipeline Stage

The table below is technology-agnostic in principle but lists representative open-source and commercial tools in each category so the architecture can be implemented immediately. Teams may substitute equivalent tools without changing the architecture.

Stage	Control Category	Representative Tools	Trigger
Plan	Threat modeling	OWASP Threat Dragon, IriusRisk	Design review / epic creation
Code	Secrets prevention	Gitleaks, TruffleHog, git-secrets	Pre-commit, pre-push
Code	SAST	CodeQL, Semgrep, SonarQube	On every pull request
Code	IDE feedback	Snyk Code, SonarLint	Real-time in editor
Build	SCA / dependency risk	Snyk Open Source, OWASP Dependency-Check, Dependabot	On build / PR
Build	SBOM generation	Syft, CycloneDX, SPDX tools	On artifact build
Build	Container image scan	Trivy, Grype, Docker Scout	On image build
Test	DAST	OWASP ZAP, Burp Suite Enterprise	Staging deploy
Test	IaC scanning	Checkov, tfsec, Terrascan	On PR touching IaC
Test	Policy as code	Open Policy Agent, Conftest, Kyverno	Pre-merge / pre-deploy gate
Release	Artifact signing & provenance	Sigstore/Cosign, in-toto, SLSA attestations	On release build
Release	Secrets management	HashiCorp Vault, AWS Secrets Manager, SOPS	Runtime injection
Deploy	Admission control	Kyverno, OPA Gatekeeper	At cluster admission
Deploy	Progressive delivery	Argo Rollouts, Flagger	Canary / blue-green rollout
Operate	Runtime security	Falco, Cilium/eBPF, Wiz Runtime	Continuous
Operate	SIEM / detection	Splunk, Elastic Security, Microsoft Sentinel	Continuous
Operate	Cloud posture (CSPM)	Wiz, Prisma Cloud, AWS Security Hub	Continuous
Govern	Compliance as code	OPA, Chef InSpec, Cloud Custodian	Scheduled + on change

3. DevSecOps Methodology

The methodology describes how teams operate the architecture day to day. It is organized around the secure software development lifecycle (SSDLC), with explicit hand-offs between development, security, and operations at each phase.

3.1 Shift-Left Security

Shift-left means moving security activities as early as possible in the lifecycle without removing later-stage verification. In practice this means:

- Security requirements are written into user stories and Definition of Ready, not added after implementation.
- Developers receive SAST and SCA findings inside the pull request, with auto-suggested fixes where available, rather than in a separate security portal.
- Secrets scanning runs client-side (pre-commit) so credentials never reach the remote repository history in the first place.
- Security champions embedded in each squad triage low-severity findings locally, escalating only true positives above an agreed severity threshold.

3.2 Threat Modeling Process

Threat modeling is performed at two cadences: once per service/system (structural) and once per significant design change (incremental).

1. Decompose the system into a data-flow diagram identifying trust boundaries, data stores, and external entities.
2. Apply STRIDE (Spoofing, Tampering, Repudiation, Information Disclosure, Denial of Service, Elevation of Privilege) to each element crossing a trust boundary.
3. Score identified threats using a DREAD or CVSS-derived rubric to prioritize mitigations.
4. Record mitigations as backlog items linked to the threat model document, stored in the repository under /docs/threat-model/.
5. Re-run the incremental process whenever a PR modifies authentication, authorization, data storage, or external integrations (flagged automatically via a CODEOWNERS rule on sensitive paths).

3.3 Secure SDLC Phase Model

Phase	Security Activity	Primary Owner	Exit Criteria
Requirements	Abuse-case & compliance requirement definition	Product + Security Champion	Security acceptance criteria added to ticket
Design	Threat model, architecture security review	Tech Lead + AppSec	Threat model approved, mitigations backlogged

Phase	Security Activity	Primary Owner	Exit Criteria
Implementation	Secure coding standards, pre-commit hooks	Developer	No secrets, lint/SAST clean on local run
Code Review	Peer review + automated SAST/SCA gate	Reviewer + Pipeline	All required checks pass, 2 approvals
Build	SBOM generation, container hardening	CI Pipeline	SBOM published, base image CVE budget met
Testing	DAST, IaC scan, policy-as-code gate	QA + Security Pipeline	No critical/high findings unresolved
Release	Artifact signing, change approval record	Release Manager	Signed artifact, provenance attached
Deploy	Admission control, progressive rollout	Platform/SRE	Policy checks pass at admission
Operate	Runtime monitoring, incident response	SRE + SecOps	Alerts triaged within SLA
Retire	Secure decommissioning, secret rotation	Owning Team	Access revoked, data disposed per policy

3.4 Compliance as Code

Rather than treating compliance frameworks (SOC 2, ISO 27001, NIST 800-53, PCI-DSS) as periodic audit exercises, controls are encoded as automated policy checks that run continuously and produce evidence artifacts automatically.

- Policies are written in Rego (OPA) or InSpec profiles and stored in a dedicated /policies repository, versioned like application code.
- Each policy links back to the specific control ID in the relevant framework (e.g., NIST 800-53 AC-6, CIS Benchmark 5.1.1) via metadata tags.
- Evidence (scan results, approval records, signed attestations) is exported automatically to the compliance evidence store on every pipeline run, eliminating manual evidence collection before audits.
- Drift from a compliant baseline (e.g., a storage bucket becoming public) triggers an automated ticket and, for critical controls, an automated remediation playbook.

3.5 Secrets Management

- No secret is ever committed to source control; pre-commit hooks and server-side push protection both enforce this as defense in depth.
- All runtime secrets are stored in a centralized secrets manager (Vault, AWS/GCP/Azure secrets services) and injected at runtime, never baked into images.
- Secrets are scoped per-environment and per-service, with automatic rotation on a defined schedule and immediate rotation on suspected exposure.
- CI/CD pipeline secrets use short-lived, OIDC-federated credentials (e.g., GitHub Actions OIDC to AWS/GCP) instead of long-lived static keys.

3.6 Container & Kubernetes Security

- Base images are pulled from an internally curated, continuously scanned registry; unscanned public images are blocked by policy at build time.
- Images are built from minimal/distroless bases where feasible to shrink the attack surface, and run as non-root with a read-only root filesystem by default.
- Kubernetes admission controllers enforce pod security standards (restricted profile), resource limits, and image provenance (signed images only) before a workload is scheduled.
- Network policies default-deny east-west traffic; explicit allow rules are required per service and are reviewed as part of the threat model.

3.7 Infrastructure as Code (IaC) Security

- All infrastructure changes flow through IaC (Terraform/Pulumi/CloudFormation); manual console changes are treated as an incident ("ClickOps" drift).
- IaC is scanned for misconfiguration (public storage, open security groups, missing encryption) on every PR, with critical findings blocking merge.
- State files are stored encrypted with restricted access; plan output is posted to the PR for human review before apply.

3.8 Continuous Monitoring & Feedback Loops

- Runtime detections (Falco, EDR, CSPM) feed back into the backlog as security debt items, closing the loop between operations and development.
- A weekly security review triages new findings by severity and age, tracking mean-time-to-remediate (MTTR) as a first-class engineering metric.
- Post-incident reviews for security events feed directly into updated threat models and new automated policy checks, so the same class of issue is caught by the pipeline going forward.

4. CI/CD Pipeline Design (GitHub Actions Implementation)

4.1 Branching Strategy

Trunk-based development is used with short-lived feature branches to keep the security feedback loop tight:

- main — always deployable; protected; every merge triggers the full pipeline and is eligible for release.
- feature/* — short-lived branches (target < 3 days) opened from main, merged via pull request only.
- release/* — optional stabilization branches cut from main for regulated releases requiring a formal change record.
- hotfix/* — emergency branches from main with an expedited but not skipped security gate (critical SAST/secrets checks still run).

4.2 Pipeline Stage Detail

#	Stage	Key Actions	Blocking?
1	Pre-commit (local)	Lint, secrets scan, unit tests	Local only
2	PR Opened	SAST (CodeQL/Semgrep), SCA, license check, IaC scan	Yes — required checks
3	Code Review	2 human approvals, CODEOWNERS for sensitive paths	Yes
4	Merge to main	Full build, unit + integration tests	Yes
5	Artifact Build	Container build, SBOM generation, image scan	Yes — critical/high CVE budget
6	Sign & Publish	Cosign signature, provenance attestation, push to registry	Yes
7	Deploy to Staging	DAST scan, policy-as-code gate (OPA)	Yes
8	Progressive Deploy to Prod	Canary rollout, automated rollback on SLO/security alert	Automated gate
9	Post-Deploy	Runtime monitoring, drift detection, evidence capture	Continuous

4.3 Sample GitHub Actions Workflow (.github/workflows/devsecops-pipeline.yml)

```
name: devsecops-pipeline

on:
  pull_request:
    branches: [ main ]
  push:
    branches: [ main ]
```

```yaml
permissions:
  contents: read
  security-events: write
  id-token: write   # for OIDC to cloud + Sigstore

jobs:
  secrets-scan:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
        with: { fetch-depth: 0 }
      - name: Gitleaks scan
        uses: gitleaks/gitleaks-action@v2

  sast:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Initialize CodeQL
        uses: github/codeql-action/init@v3
        with: { languages: javascript, python }
      - name: Run CodeQL Analysis
        uses: github/codeql-action/analyze@v3

  sca-sbom:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Generate SBOM
        uses: anchore/sbom-action@v0
        with: { format: cyclonedx-json, output-file: sbom.json }
      - name: Dependency vulnerability scan
        uses: snyk/actions/node@master
        env: { SNYK_TOKEN: "${{ secrets.SNYK_TOKEN }}" }

  iac-scan:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Checkov IaC scan
        uses: bridgecrewio/checkov-action@master
        with: { directory: infra/, framework: terraform }

  build-and-scan-image:
    needs: [secrets-scan, sast, sca-sbom, iac-scan]
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Build image
        run: docker build -t app:${{ github.sha }} .
      - name: Trivy image scan
        uses: aquasecurity/trivy-action@master
        with:
          image-ref: app:${{ github.sha }}
          severity: CRITICAL,HIGH
          exit-code: '1'
      - name: Sign image with Cosign
        run: cosign sign --yes app:${{ github.sha }}
```

```
  dast:
    needs: build-and-scan-image
    runs-on: ubuntu-latest
    steps:
      - name: OWASP ZAP baseline scan
        uses: zaproxy/action-baseline@v0.12.0
        with: { target: 'https://staging.example.com' }

  policy-gate:
    needs: [build-and-scan-image, dast]
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: OPA Conftest policy check
        run: conftest test --policy policies/ k8s/manifests/
```

4.4 Required Repository Structure

```
repo-root/
.github/
  workflows/
    devsecops-pipeline.yml
    codeql-scheduled.yml
  CODEOWNERS
  pull_request_template.md
  ISSUE_TEMPLATE/
    bug_report.md
    security_finding.md
docs/
  architecture/
  threat-model/
  runbooks/
infra/            # Terraform / IaC
policies/         # OPA / Rego / Conftest policies
src/              # Application source
tests/
sbom/             # Generated SBOM artifacts (CI output)
SECURITY.md
README.md
```

5. Governance & Accountability

5.1 RACI Matrix

R = Responsible, A = Accountable, C = Consulted, I = Informed.

Activity	Dev Team	Security Champion	AppSec/Platform	Release Mgr
Threat modeling	R	A	C	I
SAST/SCA triage	R	A	C	I
Secrets rotation	C	I	A/R	I
IaC policy authoring	C	C	A/R	I
Container hardening	R	C	A	I
Release approval	C	I	C	A/R
Incident response	R	C	A	I
Compliance evidence	I	C	A/R	I

5.2 Metrics & KPIs

Metric	Definition	Target
Mean Time to Remediate (MTTR)	Time from finding detection to verified fix, by severity	Critical < 48h, High < 7d
Vulnerability Escape Rate	% of vulnerabilities found in production vs. pre-prod	< 5%
Pipeline Security Gate Pass Rate	% of PRs passing security checks on first run	> 85%
Secrets Exposure Incidents	Count of secrets reaching remote history per quarter	0
SBOM Coverage	% of production artifacts with a published SBOM	100%
Critical CVE Backlog Age	Average age of open critical CVEs	< 15 days
Policy-as-Code Coverage	% of compliance controls automated vs. manual	> 90%
Security Champion Coverage	% of squads with an assigned champion	100%

5.3 DevSecOps Maturity Model

Level	Characteristics
1 — Ad Hoc	Security reviews are manual, late-stage, and inconsistent; no automated scanning in CI.
2 — Managed	SAST/SCA run in CI but findings triage is manual and slow; secrets scanning inconsistent.
3 — Defined	Full toolchain from this document is deployed; policies documented; MTTR tracked.
4 — Measured	Policy-as-code enforced as hard gates; SBOM + signing on every release; KPI dashboards live.
5 — Optimizing	Continuous feedback from runtime into design; automated remediation for common drift; compliance evidence fully automated.

6. GitHub Submission Process

This section operationalizes the architecture: it is the exact process a contributor follows to submit a change, and the exact configuration a repository administrator applies so the pipeline above is enforced rather than optional.

6.1 Repository Structure

```
repo-root/
.github/
  workflows/
    devsecops-pipeline.yml
    codeql-scheduled.yml
  CODEOWNERS
  pull_request_template.md
  ISSUE_TEMPLATE/
    bug_report.md
    security_finding.md
docs/
  architecture/
  threat-model/
  runbooks/
infra/            # Terraform / IaC
policies/         # OPA / Rego / Conftest policies
src/              # Application source
tests/
sbom/             # Generated SBOM artifacts (CI output)
SECURITY.md
README.md
```

6.2 Branch Protection Rules (main)

- Require a pull request before merging; require at least 2 approving reviews.
- Require review from Code Owners on paths under /infra, /policies, /src/auth, and /.github/workflows.
- Require status checks to pass before merging: secrets-scan, sast, sca-sbom, iac-scan, build-and-scan-image, dast, policy-gate.
- Require branches to be up to date before merging (no stale-base merges).
- Require signed commits (GPG or Sigstore gitsign).
- Do not allow force pushes or branch deletion on main.
- Restrict who can push directly to main to the release-automation service account only (humans always go through PR).

6.3 CODEOWNERS Example

```
# .github/CODEOWNERS
*                       @org/eng-leads
/infra/                 @org/platform-team
/policies/              @org/appsec-team
/.github/workflows/     @org/platform-team @org/appsec-team
```

```
/src/auth/              @org/appsec-team
```

6.4 Pull Request Template

```
## Summary
<!-- What does this change do and why? -->

## Security Checklist
- [ ] Threat model reviewed/updated if this touches authN, authZ, data storage, or external I/O
- [ ] No secrets or credentials added (verified by pre-commit + Gitleaks)
- [ ] New dependencies reviewed for license and known CVEs
- [ ] IaC changes reviewed via `terraform plan` output attached below
- [ ] SBOM diff reviewed for unexpected new components

## Test Evidence
<!-- Unit / integration / security scan results -->
```

6.5 Security Finding Issue Template

```
---
name: Security Finding
about: Report a vulnerability or security misconfiguration
labels: security, needs-triage
---
**Severity (CVSS or team rubric):**
**Component / Repository / Path:**
**Detection source (SAST/SCA/DAST/Manual/Runtime):**
**Description:**
**Suggested remediation:**
**Evidence (scan output, logs — redact secrets):**
```

6.6 Contributor Submission Steps

- Branch from main using the feature/<ticket-id>-short-description naming convention.
- Enable and run pre-commit hooks locally (pre-commit install) so secrets/lint issues are caught before pushing.
- Open a draft PR early to get automated checks running continuously as the change develops.
- Fill out the PR template's security checklist honestly; reviewers will reject PRs with unchecked items lacking justification.
- Resolve all blocking pipeline checks (secrets-scan, sast, sca-sbom, iac-scan, build-and-scan-image, dast, policy-gate) — a red check may not be overridden by an approval.
- Obtain the required approvals, including CODEOWNERS approval for protected paths.
- Squash-merge to main once all checks are green; the merge automatically triggers artifact build, signing, and staged deployment.

- Monitor the post-deploy canary and runtime alerts for the change; the deploying engineer owns the first 24 hours of monitoring.

7. Implementation Roadmap

Phase 1 (Weeks 1–4): Foundation

- Stand up secrets scanning (pre-commit + server-side) and SAST on all repositories.
- Establish branch protection, CODEOWNERS, and PR templates on main for pilot repositories.

Phase 2 (Weeks 5–10): Build & Test Hardening

- Add SCA/SBOM generation, container image scanning, and IaC scanning to CI.
- Stand up a centralized secrets manager and migrate hard-coded credentials.

Phase 3 (Weeks 11–16): Release & Deploy Controls

- Introduce artifact signing/provenance and policy-as-code admission control in Kubernetes.
- Enable DAST against staging environments and progressive delivery (canary) for production.

Phase 4 (Weeks 17–24): Governance & Optimization

- Automate compliance evidence capture and stand up the KPI dashboard.
- Run the first full maturity assessment and set targets for the next review cycle.

8. Appendices

Appendix A — Glossary

Term	Definition
SAST	Static Application Security Testing — analyzes source code for vulnerabilities without executing it.
DAST	Dynamic Application Security Testing — probes a running application for vulnerabilities.
SCA	Software Composition Analysis — identifies known vulnerabilities in third-party dependencies.
SBOM	Software Bill of Materials — a machine-readable inventory of software components in an artifact.
IaC	Infrastructure as Code — managing infrastructure through versioned configuration files.
OPA	Open Policy Agent — a general-purpose policy engine using the Rego language.
SLSA	Supply-chain Levels for Software Artifacts — a framework for build integrity and provenance.
CSPM	Cloud Security Posture Management — continuous scanning of cloud configuration for misconfiguration.
STRIDE	A threat modeling mnemonic: Spoofing, Tampering, Repudiation, Information Disclosure, DoS, Elevation of Privilege.

Appendix B — Reference Standards

- NIST SP 800-218 — Secure Software Development Framework (SSDF)
- OWASP Software Assurance Maturity Model (SAMM)
- OWASP Top 10 and OWASP ASVS (Application Security Verification Standard)
- CIS Benchmarks (Docker, Kubernetes, Cloud Providers)
- SLSA (Supply-chain Levels for Software Artifacts) v1.0
- ISO/IEC 27001 and SOC 2 Trust Services Criteria

Appendix C — Document Control

Version 1.0 — Initial publication. Owner: Platform Security / DevSecOps Working Group. Review cadence: Quarterly, or upon material change to toolchain or regulatory requirements.

RUBRICS FOR CALCULATION

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Complete DevSecOps Architecture Design	20%	Develops a comprehensive and technically accurate DevSecOps architecture integrating development, security, CI/CD, infrastructure and operations components	Develops a clear DevSecOps architecture containing most required components with only minor technical or integration gaps	Develops a basic DevSecOps architecture with limited integration or several missing components	Provides an incomplete or technically inaccurate DevSecOps architecture with weak component integration
Security Integration Across DevSecOps Lifecycle	30%	Effectively integrates security controls and automated security testing across planning, coding, build, test, deployment and operations stages	Integrates appropriate security controls across most DevSecOps stages with only minor omissions	Shows basic security integration but several lifecycle stages or security controls are missing	Shows limited or incorrect security integration across the DevSecOps lifecycle
Methodology and Workflow Documentation	20%	Clearly documents the complete DevSecOps methodology, workflow, tools, responsibilities and security checkpoints with strong technical justification	Documents the DevSecOps methodology and workflow clearly with minor gaps in explanation or justification	Provides basic methodology documentation with limited workflow details or technical explanation	Provides incomplete or unclear methodology documentation with weak technical reasoning
GitHub Repository and Submission Quality	20%	Submits a complete, well-organized GitHub repository containing architecture artifacts, documentation and required files with clear structure and version history	Submits a properly organized GitHub repository containing most required artifacts with minor structural or documentation issues	Submits a partially organized repository with missing artifacts, limited documentation or inconsistent file structure	GitHub submission is incomplete, poorly organized or missing essential architecture and methodology artifacts
Technical Clarity and Professional Presentation	10%	Presents the DevSecOps architecture, methodology and repository professionally with clear relationships, consistent terminology and strong technical clarity	Presents the work clearly and professionally with only minor clarity or formatting issues	Presents the work with basic clarity but noticeable inconsistencies or limited technical explanation	Presents the work poorly with unclear relationships, inconsistent terminology or insufficient technical explanation

Rubrics:

Criteria	Weightage	Q5 (10)	Total Marks (50)
Complete DevSecOps Architecture Design	25%		
Security Integration Across DevSecOps Lifecycle	30%		
Methodology and Workflow Documentation	20%		
GitHub Repository and Submission Quality	15%		
Technical Clarity and Professional Presentation	10%		
Total per Question	**100 Marks**		

100